

ABSTRACT

The rapid growth of digital financial services has made online banking systems an essential component of modern banking infrastructure. Traditional banking methods are increasingly being replaced by secure, efficient, and user-friendly digital platforms that allow customers to perform transactions anytime and anywhere. In this context, the development of a well-structured online banking database becomes crucial for managing large volumes of financial data accurately and securely. The need for this project arises from the requirement to simulate a real-world banking environment where customer information, account details, and transaction records are handled efficiently while ensuring data integrity, consistency, and security. By designing such a system, the project aims to provide a foundational understanding of how databases support critical financial operations and decision-making processes in the banking sector.

The project focuses on designing and implementing a relational database for an online banking system that includes core entities such as customers, users, accounts, and transactions. Each table is carefully structured with appropriate attributes and relationships, using primary keys to uniquely identify records and foreign keys to maintain referential integrity between tables. The database is normalized to eliminate redundancy and improve data consistency, ensuring efficient storage and retrieval of information. Customers can have multiple accounts, and each account can be associated with numerous transactions, reflecting real-world banking scenarios. Additionally, the system supports basic operations such as account creation, balance updates, fund transfers, and transaction tracking. By incorporating constraints and relationships, the project demonstrates how a robust database schema can effectively manage complex interactions within an online banking system.

The technology stack used for this project primarily includes Microsoft SQL Server for database design and implementation, along with SQL (Structured Query Language) for defining, manipulating, and querying data. Tools such as SQL Server Management Studio (SSMS) are utilized for creating tables, writing queries, and managing the database environment. The project also incorporates features such as stored procedures to automate repetitive tasks and enforce business logic within the database. Basic normalization techniques are applied to optimize the schema design, and indexing may be used to improve query performance. Overall, the chosen technologies provide a reliable and scalable platform for developing and testing the online banking database, enabling learners to gain practical experience in database management and real-world application development.

SYSTEM ANALYSIS

The current system for managing banking operations is largely based on manual processes and basic file management techniques. Customer details, account information, and transaction records are maintained using paper files and simple digital documents such as spreadsheets or text files. In this setup, bank staff manually record and update data, which makes the process time-consuming and highly dependent on human accuracy. Retrieving specific information from paper records is inefficient, especially when dealing with a large number of customers and transactions. Additionally, maintaining physical files requires significant storage space and proper organization, which can become difficult as the volume of data grows over time.

The existing system also suffers from several limitations related to data consistency, security, and redundancy. Since records are maintained manually or in separate files, there is a high risk of duplication and inconsistency of data. For example, updating customer details in one file may not reflect in another, leading to mismatched records. Security is another major concern, as paper files can be easily misplaced, damaged, or accessed by unauthorized individuals. Similarly, basic digital files lack strong access control and encryption mechanisms, making sensitive banking information vulnerable. There is also no proper mechanism to enforce relationships between different types of data, such as linking customers to their accounts or transactions, which increases the chances of errors.

Furthermore, the existing system does not support real-time processing or efficient reporting. Generating reports, such as account summaries or transaction histories, requires manual compilation of data, which is both time-consuming and prone to errors. The absence of automation makes it difficult to handle high volumes of transactions and limits the scalability of the system. As a result, the current approach is not suitable for modern banking requirements, where speed, accuracy, and security are critical. These limitations highlight the need for a computerized database system that can streamline operations, reduce manual effort, and ensure reliable data management.

DRAWBACKS OF EXISTING SYSTEM

The existing system, which relies on paper files and basic digital documents, has several significant drawbacks that affect efficiency and reliability. One of the major issues is the high dependency on manual work, which increases the chances of human error. Data entry mistakes, misplaced records, and incorrect updates are common, leading to inconsistencies in customer, account, and transaction information. Additionally, searching for specific records in physical files or scattered digital documents is time-consuming and inefficient, especially as the volume of data increases.

Another major drawback is the lack of proper data security and integrity. Paper files can be easily damaged, lost, or accessed by unauthorized individuals, while simple digital files do not provide strong security mechanisms such as authentication or encryption. There is also no effective way to maintain relationships between different data entities, which can result in duplicate or inconsistent data. Since updates are not automatically synchronized across records, maintaining accurate and up-to-date information becomes difficult.

The system also lacks scalability and automation, making it unsuitable for modern banking needs. Handling a large number of transactions manually slows down operations and reduces productivity. Generating reports such as transaction histories or account summaries requires manual effort, increasing the likelihood of errors and delays. Overall, the absence of a centralized and structured database limits the system's ability to provide fast, accurate, and secure services, highlighting the need for an improved computerized solution.

PROPOSED SYSTEM

The proposed system is a computerized online banking database designed to replace the existing manual and file-based approach with a centralized, efficient, and secure solution. This system organizes all banking data into structured tables such as customers, users, accounts, and transactions, ensuring that information is stored systematically and can be accessed easily when needed. By using a relational database model, the system establishes clear relationships between different entities, allowing seamless integration of data and reducing redundancy. The introduction of this system significantly improves the speed and accuracy of banking operations, enabling faster data retrieval and real-time updates.

In the proposed system, data integrity and consistency are maintained through the use of primary keys, foreign keys, and normalization techniques. These features ensure that each record is uniquely identified and properly linked with related data, minimizing duplication and errors. The system also incorporates constraints and validation rules to maintain accuracy during data entry and updates. Additionally, stored procedures and queries are used to automate common banking operations such as account creation, fund transfers, balance checks, and transaction tracking. This automation reduces manual effort and enhances operational efficiency, making the system more reliable and user-friendly.

Security and scalability are key strengths of the proposed system. Unlike the existing system, it includes authentication mechanisms and controlled access to protect sensitive financial data from unauthorized users. The centralized database allows for easy backup and recovery, ensuring data safety in case of system failures. Furthermore, the system supports efficient report generation, enabling quick access to account summaries and transaction histories. As the number of users and transactions grows, the database can be scaled accordingly without compromising performance. Overall, the proposed system provides a robust, secure, and efficient solution that meets the demands of modern banking operations.

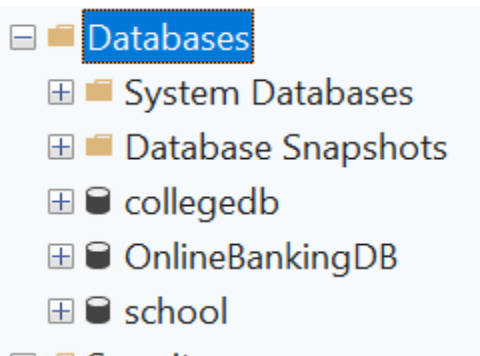
ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers a wide range of advantages over the existing manual and file-based approach, primarily in terms of efficiency and accuracy. By automating banking operations through a structured database, the system significantly reduces the need for manual data entry and processing. This minimizes human errors such as incorrect entries, duplication, and data inconsistencies. Information can be retrieved quickly using queries, which saves time and improves overall productivity. Tasks that previously required hours, such as searching for customer records or transaction details, can now be completed within seconds.

Another key advantage of the proposed system is improved data integrity and security. The use of primary keys and foreign keys ensures that all data is properly linked and maintained without redundancy. Constraints and validation rules help enforce accuracy during data entry, preventing invalid or incomplete records from being stored. In addition, the system provides controlled access through authentication mechanisms, ensuring that only authorized users can view or modify sensitive banking information. Backup and recovery features further enhance data safety, protecting against data loss due to system failures or unexpected issues.

The proposed system is also highly scalable and supports better decision-making through efficient reporting. As the number of customers and transactions increases, the database can handle large volumes of data without affecting performance. Automated report generation allows bank staff to quickly produce account summaries, transaction histories, and other important reports, aiding in analysis and decision-making. Overall, the system enhances reliability, reduces operational costs, improves service quality, and provides a strong foundation for future expansion and integration with advanced banking technologies.

CREATE DATABASE OnlineBankingDB;



```

CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY IDENTITY(1,1),
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Phone VARCHAR(15),
    Address VARCHAR(255),
    CreatedAt DATETIME DEFAULT GETDATE()
);

```

Column Name	Data Type	Allow Nulls
CustomerID	int	☐
FullName	varchar(100)	☐
Email	varchar(100)	☐
Phone	varchar(15)	☒
Address	varchar(255)	☒
CreatedAt	datetime	☒
		☐

Results		Messages				
	CustomerID	FullName	Email	Phone	Address	CreatedAt
1	1	Bharath Kumar	bharath@gmail.com	9876543210	Kerala	2026-04-26 11:49:07.137
2	2	Rahul Nair	rahul@gmail.com	9123456780	Trivandrum	2026-04-26 11:49:07.137

```

CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),

```

```

CustomerID INT,
Username VARCHAR(50) UNIQUE NOT NULL,
PasswordHash VARCHAR(255) NOT NULL,
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

```

Column Name	Data Type	Allow Nulls
UserID	int	☐
CustomerID	int	☒
Username	varchar(50)	☐
PasswordHash	varchar(255)	☐
		☐

Results

Messages

	UserID	CustomerID	Username	PasswordHash
1	1	1	bharath	hashed_password1
2	2	2	rahul	hashed_password2

```

CREATE TABLE Accounts (
    AccountID INT PRIMARY KEY IDENTITY(1,1),
    CustomerID INT,
    AccountType VARCHAR(20),
    Balance DECIMAL(15,2) DEFAULT 0,
    CreatedAt DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

```

Column Name	Data Type	Allow Nulls
AccountID	int	☐
CustomerID	int	☒
AccountType	varchar(20)	☒
Balance	decimal(15, 2)	☒
CreatedAt	datetime	☒
		☐

	AccountID	CustomerID	AccountType	Balance	CreatedAt
1	1	1	Savings	10500.00	2026-04-26 12:07:10.313
2	2	1	Current	11000.00	2026-04-26 12:07:10.313
3	3	2	Savings	500.00	2026-04-26 12:07:10.313
4	4	2	Current	8000.00	2026-04-26 12:07:10.313
5	5	1	Savings	0.00	2026-04-26 14:01:44.040
6	6	2	Savings	0.00	2026-04-26 14:01:44.040
7	7	1	Savings	0.00	2026-04-27 10:01:45.903
8	8	2	Savings	0.00	2026-04-27 10:01:58.443

```

CREATE TABLE Transactions (
    TransactionID INT PRIMARY KEY IDENTITY(1,1),
    FromAccountID INT NULL,
    ToAccountID INT NULL,
    Amount DECIMAL(15,2),
    TransactionType VARCHAR(20),
    TransactionDate DATETIME DEFAULT GETDATE()
);

```

Column Name	Data Type	Allow Nulls
TransactionID	int	☐
FromAccountID	int	☒
ToAccountID	int	☒
Amount	decimal(15, 2)	☒
TransactionType	varchar(20)	☒
TransactionDate	datetime	☒
		☐

Results		Messages				
	TransactionID	FromAccountID	ToAccountID	Amount	TransactionType	TransactionDate
1	1	NULL	1	5000.00	Deposit	2026-04-26 13:52:49.477
2	2	1	NULL	1000.00	Withdraw	2026-04-26 13:52:49.477
3	3	1	3	500.00	Transfer	2026-04-26 13:52:49.477
4	4	NULL	2	10000.00	Deposit	2026-04-26 13:52:49.477
5	5	2	4	2000.00	Transfer	2026-04-26 13:52:49.477
6	6	4	NULL	1500.00	Withdraw	2026-04-26 13:52:49.477
7	7	NULL	1	5000.00	Deposit	2026-04-26 14:02:14.867
8	8	1	NULL	1000.00	Withdraw	2026-04-26 14:02:38.957
9	9	1	2	500.00	Transfer	2026-04-26 14:02:59.193
10	10	NULL	1	5000.00	Deposit	2026-04-27 10:02:05.240
11	11	1	NULL	1000.00	Withdraw	2026-04-27 10:02:15.150
12	12	1	2	500.00	Transfer	2026-04-27 10:02:22.520

STORED PROCEDURE

CREATE PROCEDURE CreateAccount

@CustomerID INT,

@AccountType VARCHAR(20)

AS

BEGIN

INSERT INTO Accounts (CustomerID, AccountType)

VALUES (@CustomerID, @AccountType);

END;

```

GO

CREATE PROCEDURE DepositMoney
    @AccountID INT,
    @Amount DECIMAL(15,2)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE Accounts
        SET Balance = Balance + @Amount
        WHERE AccountID = @AccountID;

        INSERT INTO Transactions (ToAccountID, Amount, TransactionType)
        VALUES (@AccountID, @Amount, 'Deposit');

        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK;
    END CATCH
END;
GO

CREATE PROCEDURE WithdrawMoney
    @AccountID INT,
    @Amount DECIMAL(15,2)
AS
BEGIN
    DECLARE @Balance DECIMAL(15,2);

    SELECT @Balance = Balance FROM Accounts WHERE AccountID = @AccountID;

```

```

IF @Balance < @Amount
BEGIN
    RAISERROR ('Insufficient Balance', 16, 1);
    RETURN;
END

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - @Amount
    WHERE AccountID = @AccountID;

    INSERT INTO Transactions (FromAccountID, Amount, TransactionType)
    VALUES (@AccountID, @Amount, 'Withdraw');

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
END CATCH
END;
GO

CREATE PROCEDURE TransferMoney
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(15,2)
AS
BEGIN
    DECLARE @Balance DECIMAL(15,2);

    SELECT @Balance = Balance FROM Accounts WHERE AccountID = @FromAccount;

```

```

IF @Balance < @Amount
BEGIN
    RAISERROR ('Insufficient Balance', 16, 1);
    RETURN;
END

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - @Amount
    WHERE AccountID = @FromAccount;

    UPDATE Accounts
    SET Balance = Balance + @Amount
    WHERE AccountID = @ToAccount;

    INSERT INTO Transactions (FromAccountID, ToAccountID, Amount, TransactionType)
    VALUES (@FromAccount, @ToAccount, @Amount, 'Transfer');

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
END CATCH
END;
GO

```

```

CREATE PROCEDURE GetTransactionHistory
    @AccountID INT
AS BEGIN
    SELECT *
    FROM Transactions
    WHERE FromAccountID = @AccountID
        OR ToAccountID = @AccountID
    ORDER BY TransactionDate DESC;
END;
GO

CREATE VIEW AccountSummary AS
SELECT
    C.FullName,
    A.AccountID,
    A.AccountType,
    A.Balance
FROM Customers C
JOIN Accounts A ON C.CustomerID = A.CustomerID;
GO

CREATE TRIGGER trg_AfterTransaction
ON Transactions
AFTER INSERT
AS
BEGIN
    PRINT 'Transaction Completed Successfully';
END;
GO

```

```

EXEC CreateAccount 1, 'Savings';
EXEC CreateAccount 2, 'Savings';
EXEC DepositMoney 1, 5000;
EXEC WithdrawMoney 1, 1000;
EXEC TransferMoney 1, 2, 500;
EXEC GetTransactionHistory 1;
SELECT * FROM AccountSummary;

```

```

188  GO
189  EXEC CreateAccount 1, 'Savings';
190  EXEC CreateAccount 2, 'Savings';
191  EXEC DepositMoney 1, 5000;
192  EXEC WithdrawMoney 1, 1000;
193  EXEC TransferMoney 1, 2, 500;
194  EXEC GetTransactionHistory 1;
195  SELECT * FROM AccountSummary;
196  select * from Accounts;
197  select AccountID,Balance from Accounts;
198
199  select * from Customers;
200  select * from Accounts;
201  select*from Customers;
202  select*from Transactions;

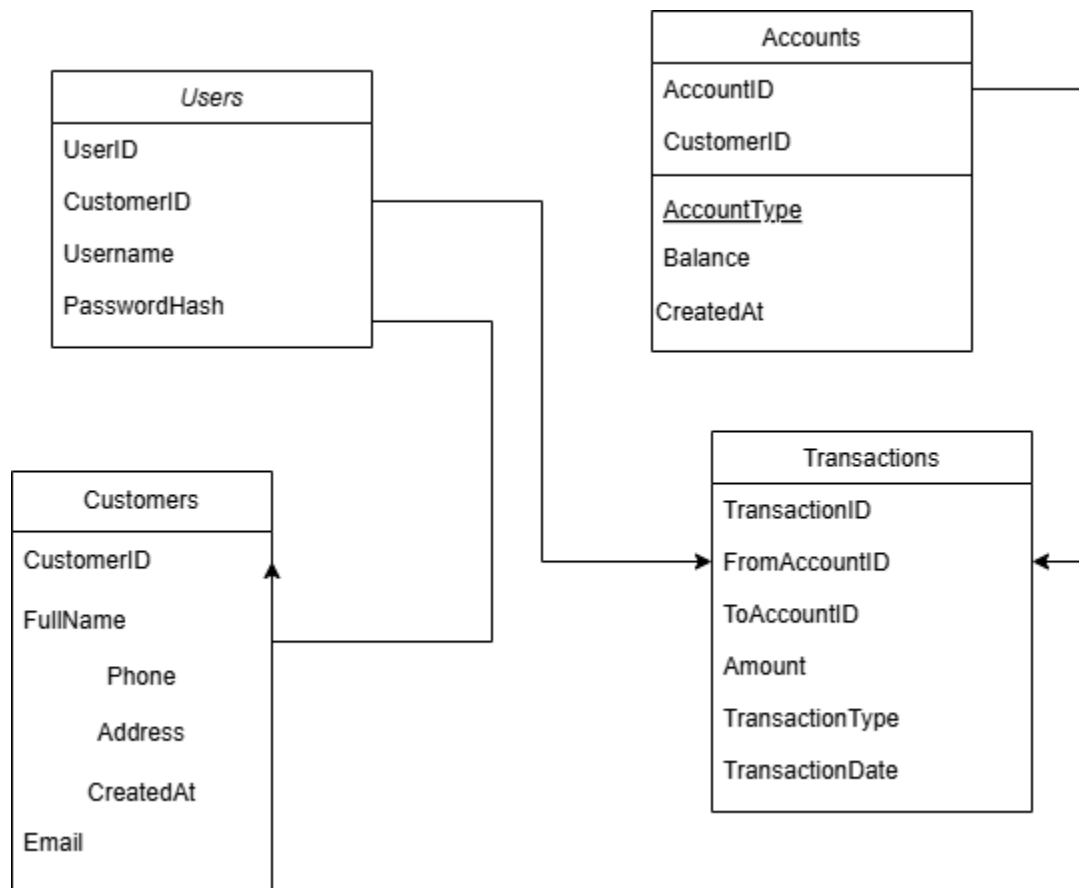
```

80 % 3 0 ↑ ↓

Results Messages

	TransactionID	FromAccountID	ToAccountID	Amount	TransactionType	TransactionDate
4	7	2	1	5000.00	Deposit	2026-04-27 10:14:14.460
5	8	1	3	1000.00	Withdraw	2026-04-27 10:14:14.460
6	9	1	3	500.00	Transfer	2026-04-27 10:14:14.460
7	10	1	2	10000...	Deposit	2026-04-27 10:14:14.460
8	1	NULL	1	5000.00	Deposit	2026-04-27 09:59:28.960
9	2	1	NULL	1000.00	Withdraw	2026-04-27 09:59:28.960
10	3	1	3	500.00	Transfer	2026-04-27 09:59:28.960

DATA FLOW DIAGRAM



USE CASE DIAGRAM

